Final Project

```
library(ggplot2)
```

Project Prompt

You and your team of data scientists were hired by a company to gain insights on the dataset in your assigned prompt, and to communicate these insights with the company's CEO, CMO, and Director of Data Science.

1. Perform any necessary data pre-processing and cleaning, and document your steps. Depending on the prompt you are assigned, this may involve transforming variables, creating new variables, and/or merging data frames. In particular, you may need to make some decisions on how to handle missing data, such as removing rows or columns with a significant amount of missing observations, creating an "unknown" category, or replacing/imputing the missing values. You do not need to develop a rigorous process or cite references, but please briefly justify your choices.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ forcats   1.0.0     ✓ stringr   1.5.1
## ✓ lubridate 1.9.4     ✓ tibble    3.3.0
## ✓ purrr     1.1.0     ✓ tidyr     1.3.1
## ✓ readr     2.1.5
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
e errors
```

```
library(readr)
questionnaire <- read_csv("C:/Users/Hp/Desktop/NHANES_Rebecca/questionnaire.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 10175 Columns: 953
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr   (2): SMDUPCA, SMD100BR
## dbl (878): SEQN, ACD011A, ACD011C, ACD040, ACD110, ALQ101, ALQ110, ALQ120Q, ...
## lgl  (73): ACD011B, CSQ120A, CDQ009A, CDQ009G, CDQ009H, DIQ175V, DIQ175W, DU...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
covariate_df <- read_csv("C:/Users/Hp/Desktop/NHANES_Rebecca/covariate_df.rData")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 89256 Columns: 1
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr (1): RDX3
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
outcomes_only <- dplyr::select(questionnaire, SEQN, DPQ020, DPQ100)

load("C:/Users/Hp/Documents/covariate_df.rData")
full_df <- full_join(covariate_df, outcomes_only)
```

```
## Joining with `by = join_by(SEQN)`
```

```
#looking at the amount of missingness in outcome
sum(is.na(full_df$DPQ020))
```

```
## [1] 4779
```

```
#looking at total size of full (joined) data frame
dim(full_df)
```

```
## [1] 10175   862
```

```
#want to drop rows with missing outcome data
bad_missing_indices <- which(is.na(full_df$DPQ020))
modified_full_df <- full_df[-bad_missing_indices,]

#want to ignore values for depression that are not 1 or 2
not_1_or_2_indices <- which(modified_full_df$DPQ020 != 1 & modified_full_df$DPQ020 != 2)
#use DeMorgan's Law: not (1 or 2) is logically equivalent to not 1 and not 2
modified2_full_df <- modified_full_df[-not_1_or_2_indices,]
```

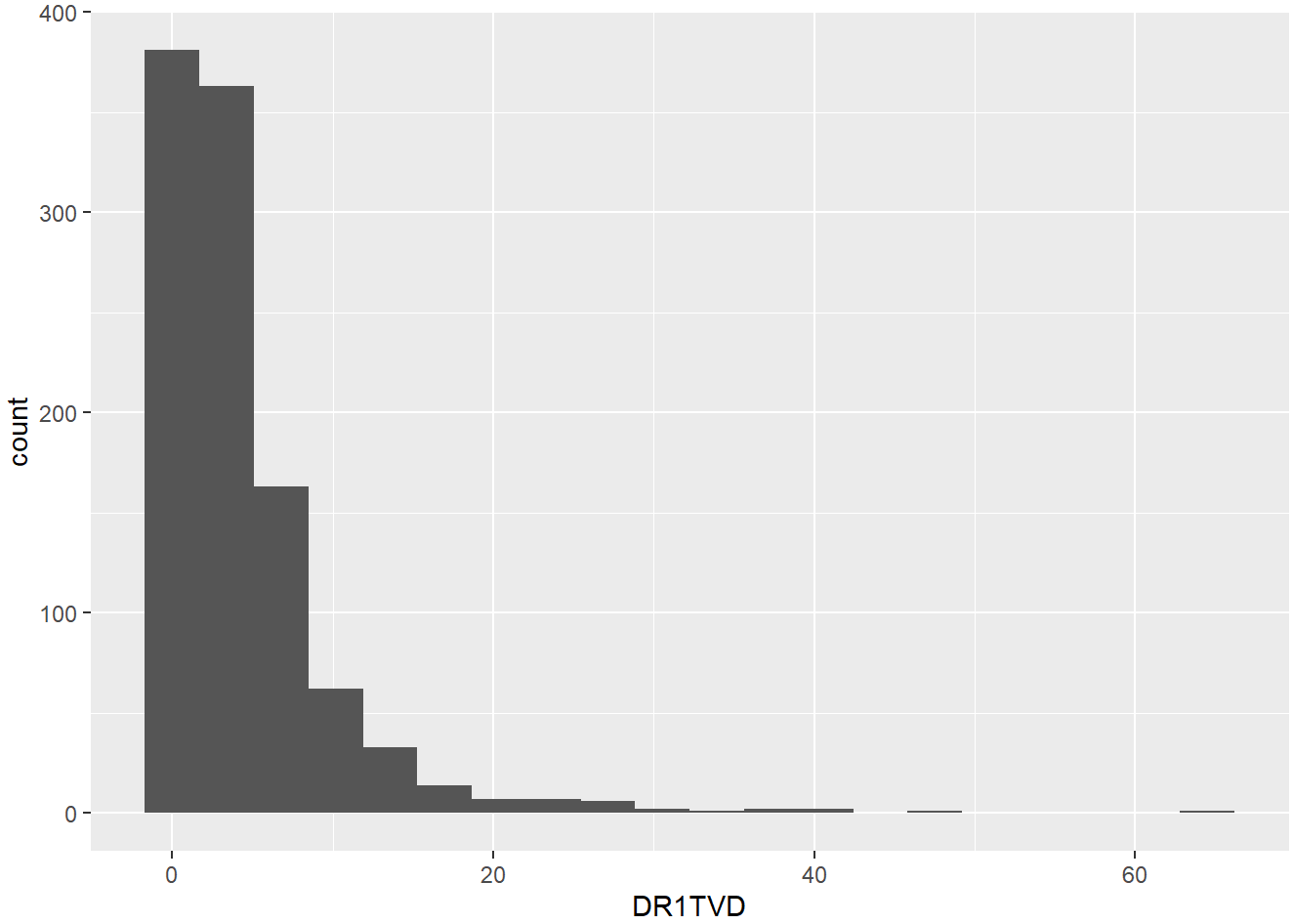
After doing this data pre-processing, only rows with complete outcome data on depression have been kept. Note that I did perform median imputation later in the code (Rmd file), after realizing that k nearest neighbors model would not run with missing data. I chose median imputation rather than mean imputation because the variables I chose have histograms that do not support the normality assumption.

2. Include a “Table 1” of summary statistics for the variables you end up using in your models.

```
#need to know which variables are normal and which are not normally distributed (for Table One t
o be correctly displayed with median and IQR)

#starting with DR1TVD
hist_DR1TVD <- modified2_full_df %>% ggplot(aes(DR1TVD)) +
  geom_histogram(bins = 20)
hist_DR1TVD
```

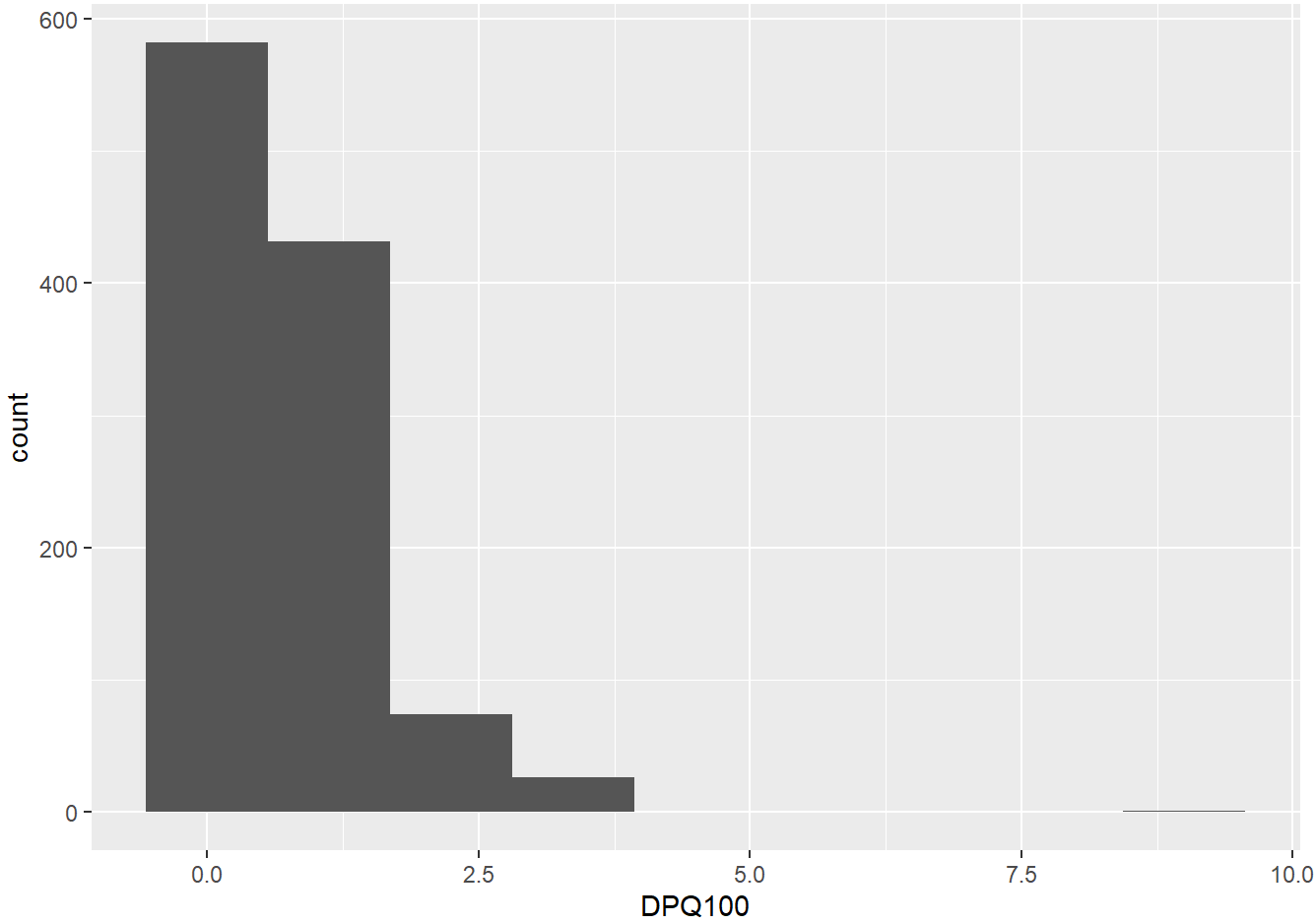
```
## Warning: Removed 72 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
#conclusion: not normal

#potentially including DPQ100
hist_DPQ100 <- modified2_full_df %>% ggplot(aes(DPQ100)) +
  geom_histogram(bins = 9)
hist_DPQ100
```

```
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

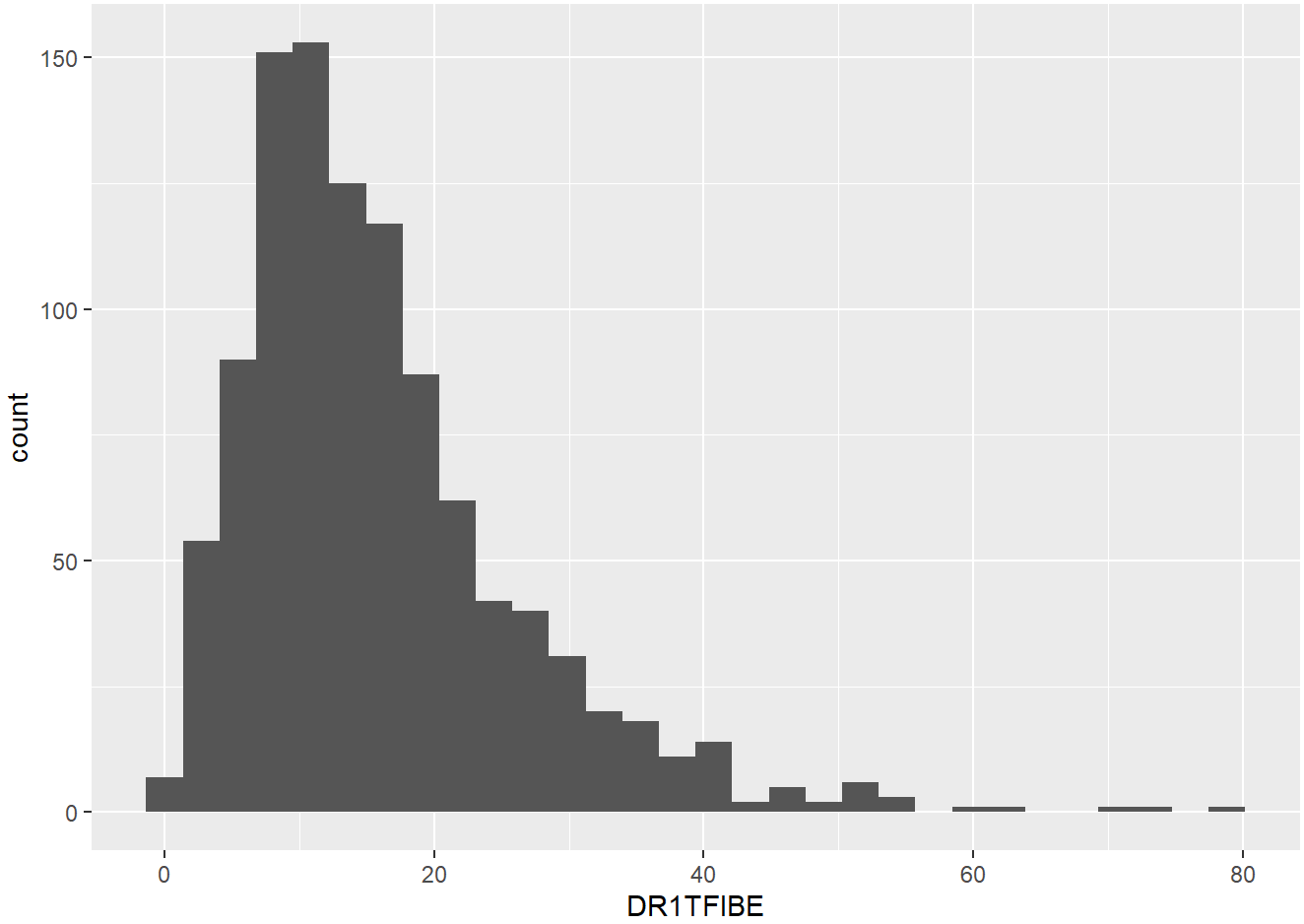



```
#conclusion: not normal

#next up: DR1TFIBE
hist_DR1TFIBE <- modified2_full_df %>% ggplot(aes(DR1TFIBE)) +
  geom_histogram()
hist_DR1TFIBE
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

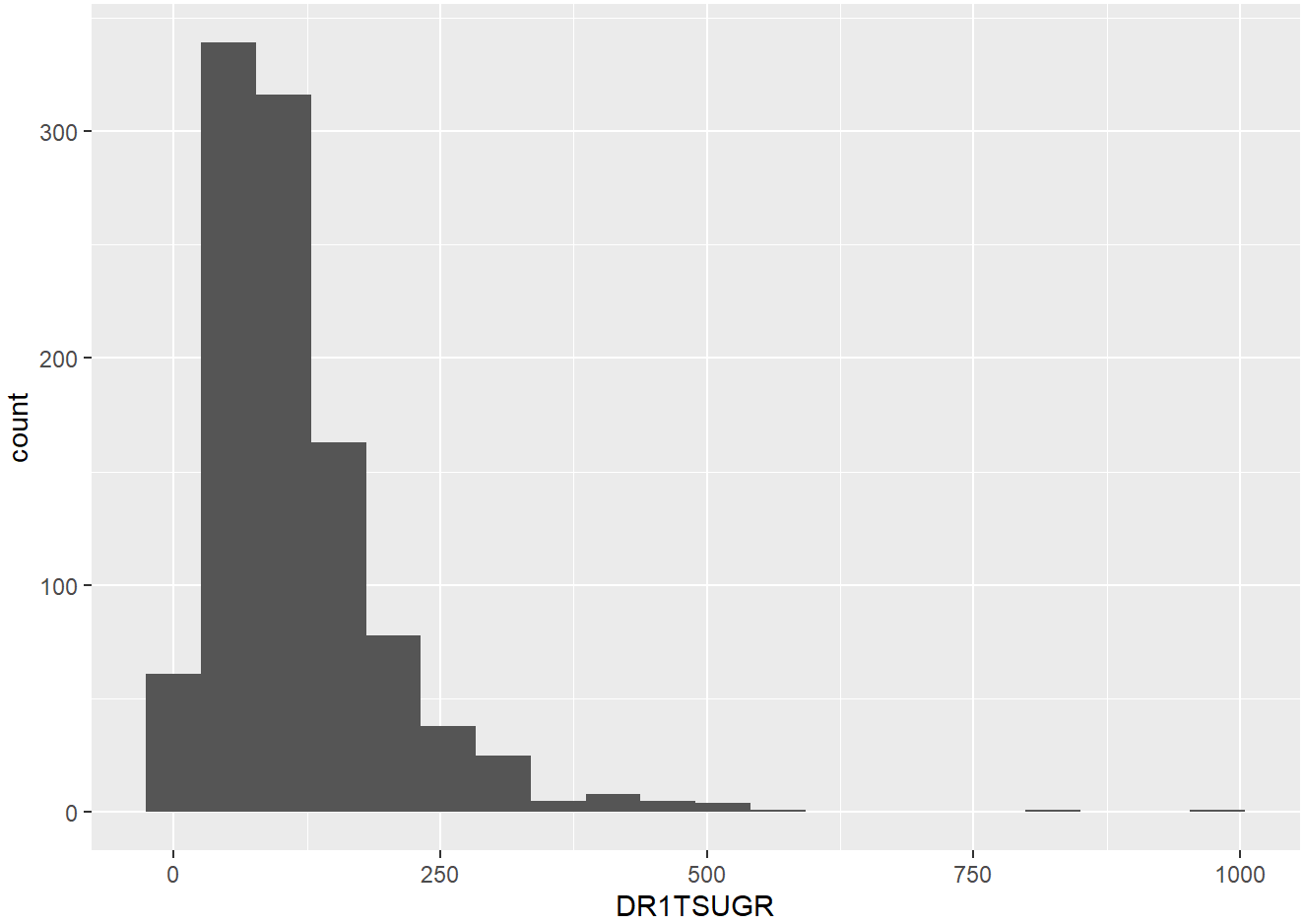
```
## Warning: Removed 72 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
#conclusion: not normal

#next up: DR1TSUGR
hist_DR1TSUGR <- modified2_full_df %>% ggplot(aes(DR1TSUGR)) +
  geom_histogram(bins = 20)
hist_DR1TSUGR
```

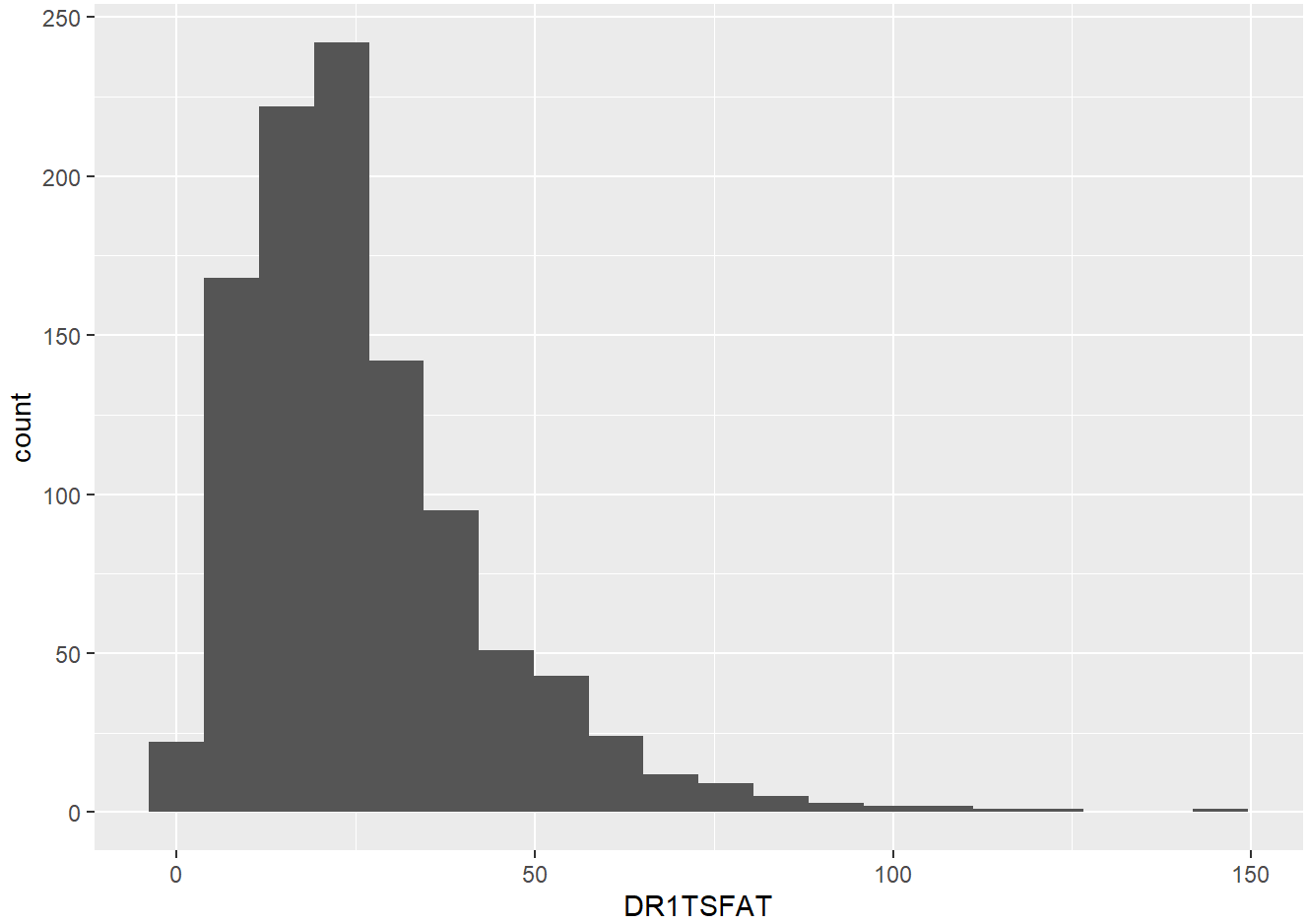
```
## Warning: Removed 72 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
#conclusion: not normal

#next up: DR1TSFAT
hist_DR1TSFAT <- modified2_full_df %>% ggplot(aes(DR1TSFAT)) +
  geom_histogram(bins = 20)
hist_DR1TSFAT
```

```
## Warning: Removed 72 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

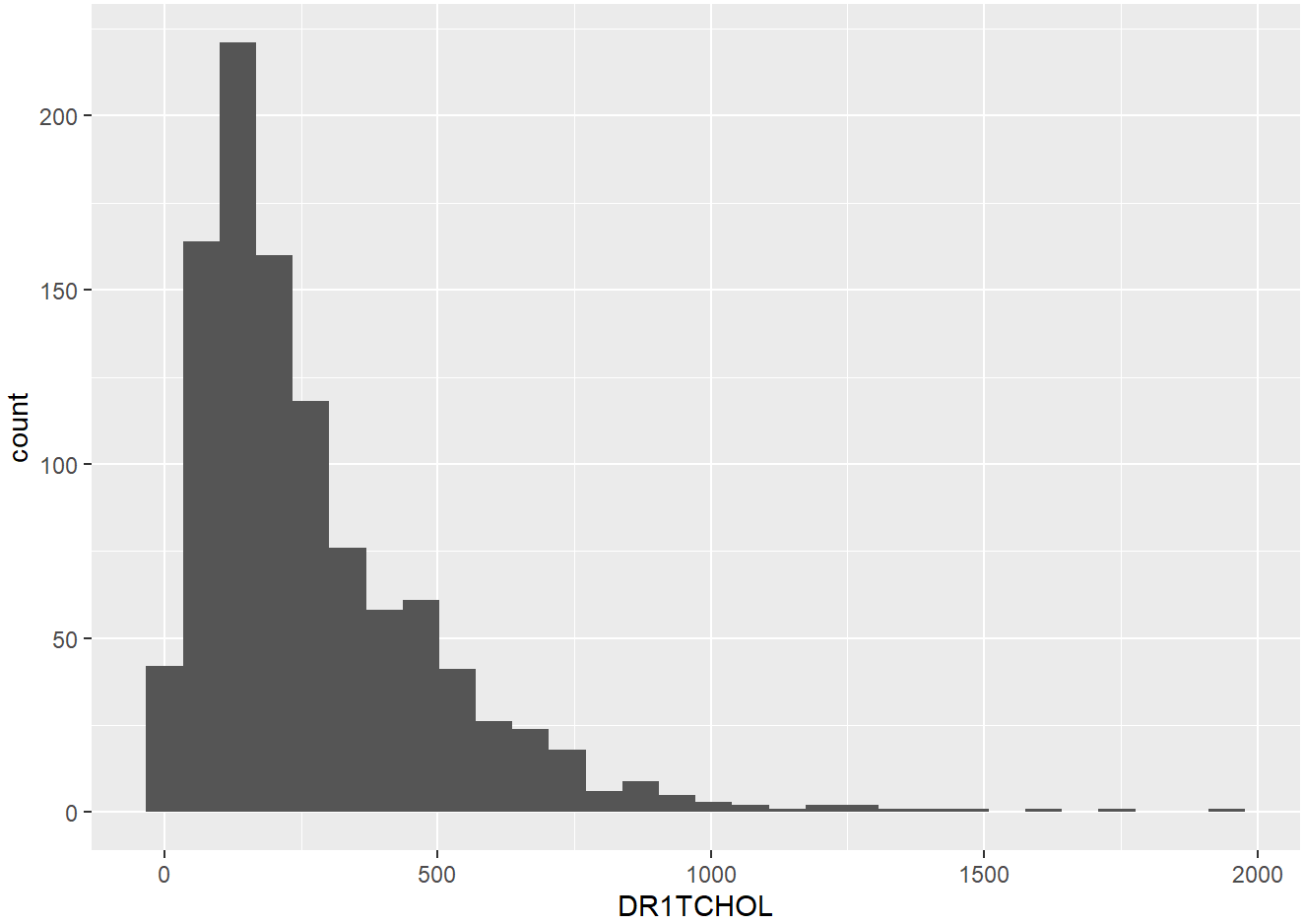



```
#conclusion: not normal

#next up: DR1TCHOL
hist_DR1TCHOL <- modified2_full_df %>% ggplot(aes(DR1TCHOL)) +
  geom_histogram()
hist_DR1TCHOL
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
## Warning: Removed 72 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
#conclusion: not normal

#none of my variables are normally distributed
```

```
library(tableone)
```

```
## Warning: package 'tableone' was built under R version 4.5.2
```

```
my_vars <- c("DR1TVD", "DPQ100", "DR1TFIBE", "DR1TSUGR", "DR1TSFAT", "DR1TCHOL")

t1 <- CreateTableOne(vars = my_vars, strata = c("DPQ020"), data = modified2_full_df)
t1
```

```
##                       Stratified by DPQ020
##                        1               2               p      test
##   n                       903             214
##   DR1TVD (mean (SD))     4.53 (5.95)     3.70 (4.10)    0.069
##   DPQ100 (mean (SD))     0.53 (0.67)     0.88 (1.01)   <0.001
##   DR1TFIBE (mean (SD))  16.00 (10.28)   15.67 (10.65)   0.685
##   DR1TSUGR (mean (SD)) 114.60 (86.47)  120.43 (102.89)  0.417
##   DR1TSFAT (mean (SD))  26.09 (17.29)   26.53 (19.20)   0.755
##   DR1TCHOL (mean (SD)) 273.05 (229.80) 280.73 (250.38)  0.682
```

```
summary(t1)
```

```
##
##      ### Summary of continuous variables ###
##
## DPQ020: 1
##            n miss p.miss  mean    sd median p25 p75 min  max skew kurt
## DR1TVD   903   49    5.4   4.5   5.9      3   1   6 0.0   64    4   22
## DPQ100   903    1    0.1   0.5   0.7      0   0   1 0.0    3    1    2
## DR1TFIBE 903   49    5.4  16.0  10.3     14   9  20 0.0   79    2    5
## DR1TSUGR 903   49    5.4 114.6  86.5     97  59 144 0.6  979    3   16
## DR1TSFAT 903   49    5.4  26.1  17.3     22  14  34 0.5  146    2    6
## DR1TCHOL 903   49    5.4 273.1 229.8    202 118 361 0.0 1944    2    8
## ------------------------------------------------------------
## DPQ020: 2
##            n miss p.miss  mean  sd median   p25 p75  min  max skew kurt
## DR1TVD   214   23   10.7   3.7   4      3   0.8   5  0.0   25    2    6
## DPQ100   214    1    0.5   0.9   1      1   0.0   1  0.0    9    3   18
## DR1TFIBE 214   23   10.7  15.7  11     13   7.8  20  0.5   53    1    1
## DR1TSUGR 214   23   10.7 120.4 103     88  50.7 160 11.3  836    3   12
## DR1TSFAT 214   23   10.7  26.5  19     23  12.8  35  0.6   99    1    2
## DR1TCHOL 214   23   10.7 280.7 250    198 115.0 412  6.0 1585    2    6
##
## p-values
##               pNormal   pNonNormal
## DR1TVD   6.858764e-02 1.716460e-01
## DPQ100   1.806033e-09 3.414180e-07
## DR1TFIBE 6.850873e-01 3.423966e-01
## DR1TSUGR 4.169708e-01 6.530567e-01
## DR1TSFAT 7.550409e-01 7.370612e-01
## DR1TCHOL 6.815468e-01 9.100486e-01
##
## Standardize mean differences
##              1 vs 2
## DR1TVD   0.16157320
## DPQ100   0.40278162
## DR1TFIBE 0.03210284
## DR1TSUGR 0.06133379
## DR1TSFAT 0.02413198
## DR1TCHOL 0.03194824
```

```
kableone(t1, nonnormal = my_vars)
```

	1	2	p	test
n	903	214		
DR1TVD (median [IQR])	2.85 [1.00, 5.88]	2.60 [0.85, 4.75]	0.172	nonnorm
DPQ100 (median [IQR])	0.00 [0.00, 1.00]	1.00 [0.00, 1.00]	<0.001	nonnorm

	1	2	p	test
DR1TFIBE (median [IQR])	13.70 [9.10, 20.38]	13.30 [7.75, 20.30]	0.342	nonnorm
DR1TSUGR (median [IQR])	97.25 [59.47, 144.36]	87.65 [50.68, 160.13]	0.653	nonnorm
DR1TSFAT (median [IQR])	22.49 [14.21, 33.67]	22.93 [12.76, 34.55]	0.737	nonnorm
DR1TCHOL (median [IQR])	202.00 [118.50, 361.00]	198.00 [115.00, 412.00]	0.910	nonnorm

3. Make and interpret 2-6 visualizations to help you understand the relationships between the variables in your dataset. We highly encourage you to explore the data on your own, but when preparing your response to this question, please be parsimonious in your plots and select visualizations that help you tell a story about the data. If you need to make additional plots to support your responses to the other questions (e.g., to motivate data cleaning or modeling choices), that's fine.

This (hypothetical) group of scientists is interested in learning about the relationship between nutritional status and depression as an outcome.

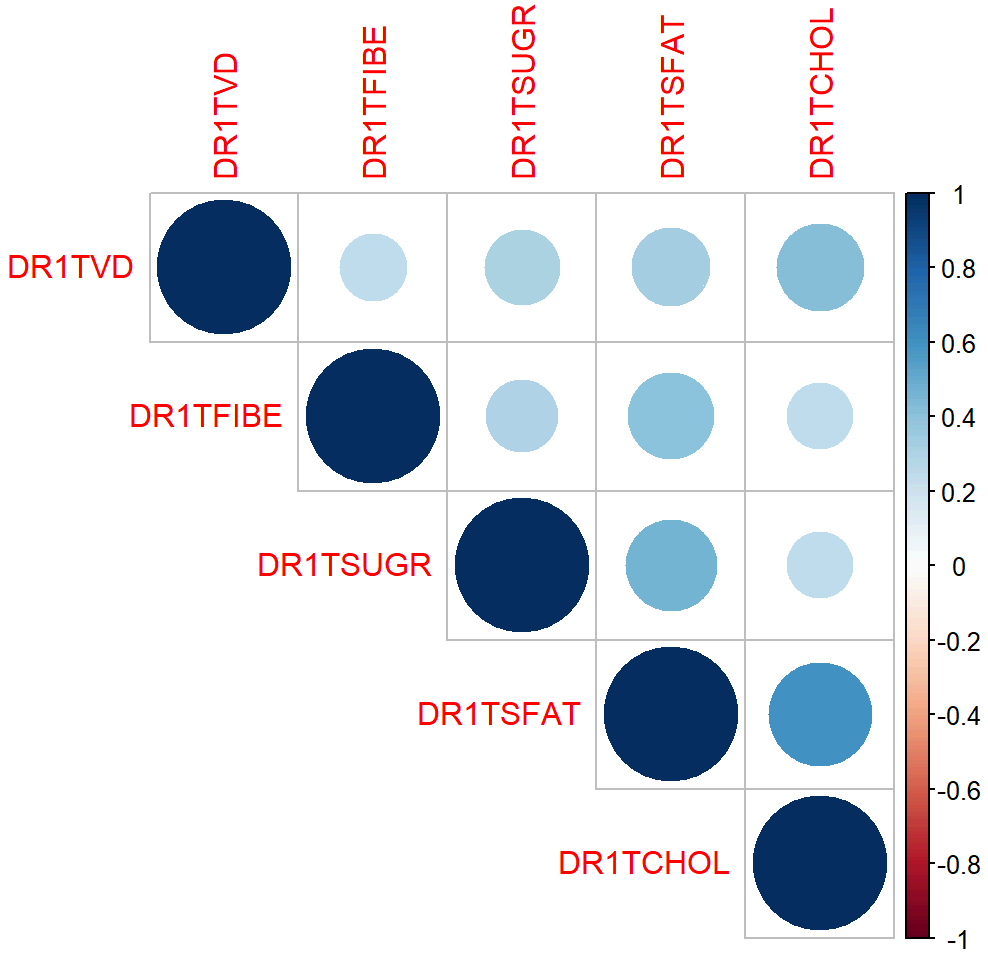
Vitamin D deficiency is commonly known to contribute to depression, and we would like to know if the NHANES data set supports this conclusion. We are also looking at fiber, sugar, saturated fat, and cholesterol consumption.

```
#install.packages("corrplot")
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.5.3
```

```
## corrplot 0.95 loaded
```

```
my_vars_corr <- c("DR1TVD", "DR1TFIBE", "DR1TSUGR", "DR1TSFAT", "DR1TCHOL")
data_subset <- modified2_full_df[,my_vars_corr]
corr_matrix <- cor(data_subset, method = "spearman", use = "pairwise.complete.obs")
corrplot(corr_matrix, type = "upper")
```

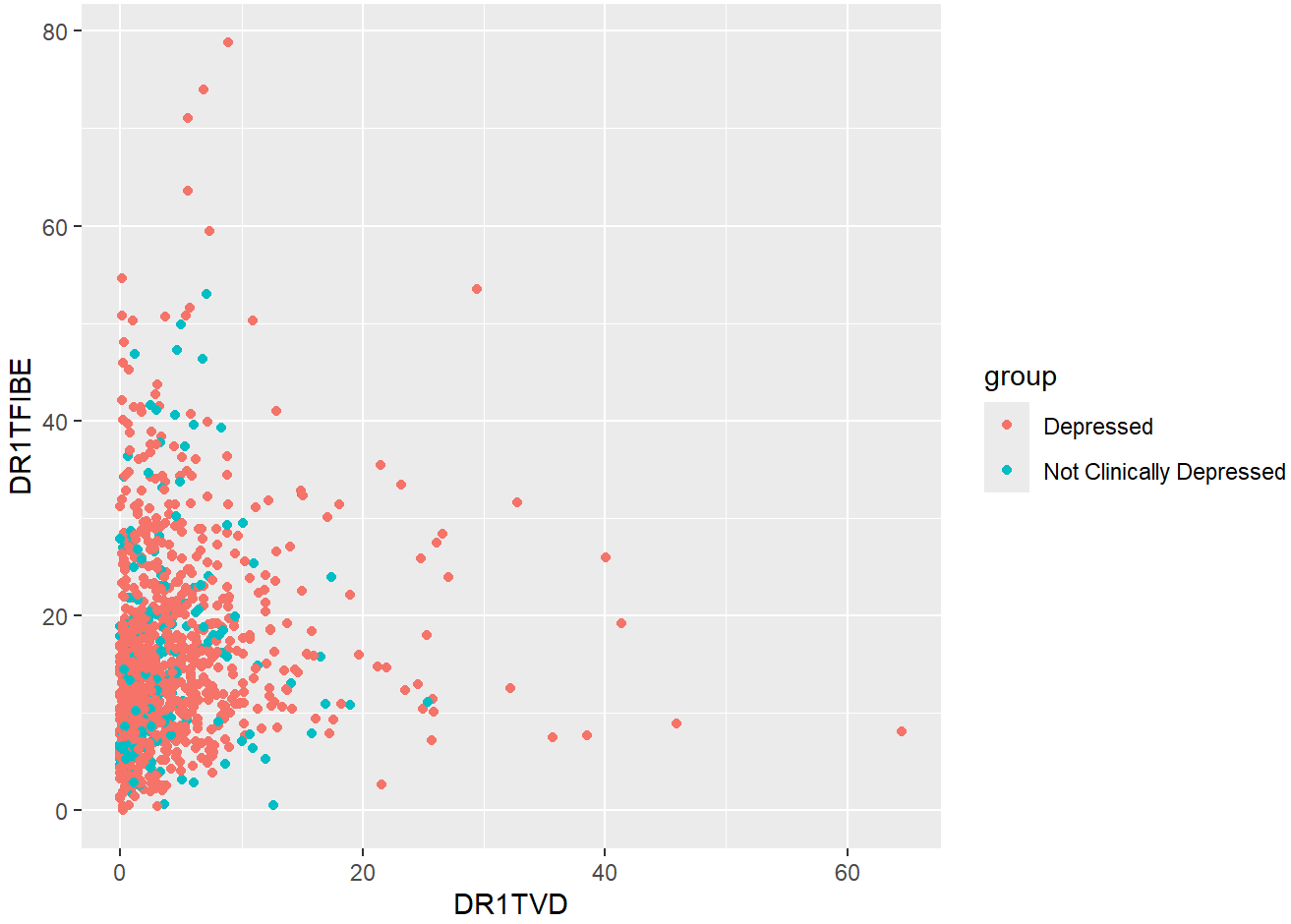


From this visualization, it appears all variables are positively correlated with each other. In other words, as a person consumes more vitamin D, fiber, sugar, saturated fat, or cholesterol, they on average consume more of the other dietary components as well.

Let us examine the relationship between vitamin D consumption and depression, as evidenced by the variable “DPQ020.” Scientists and physicians believe that vitamin D deficiency can contribute to depression, either causing it or worsening symptoms.

```
modified3_full_df <- modified2_full_df %>%
  mutate(group = ifelse(DPQ020 == 1, "Depressed", "Not Clinically Depressed"))

vit_D_and_fiber_and_depression_plot <- modified3_full_df %>%
  ggplot(aes(x = DR1TVD, y = DR1TFIBE, color = group)) +
  geom_point(na.rm = TRUE)
vit_D_and_fiber_and_depression_plot
```

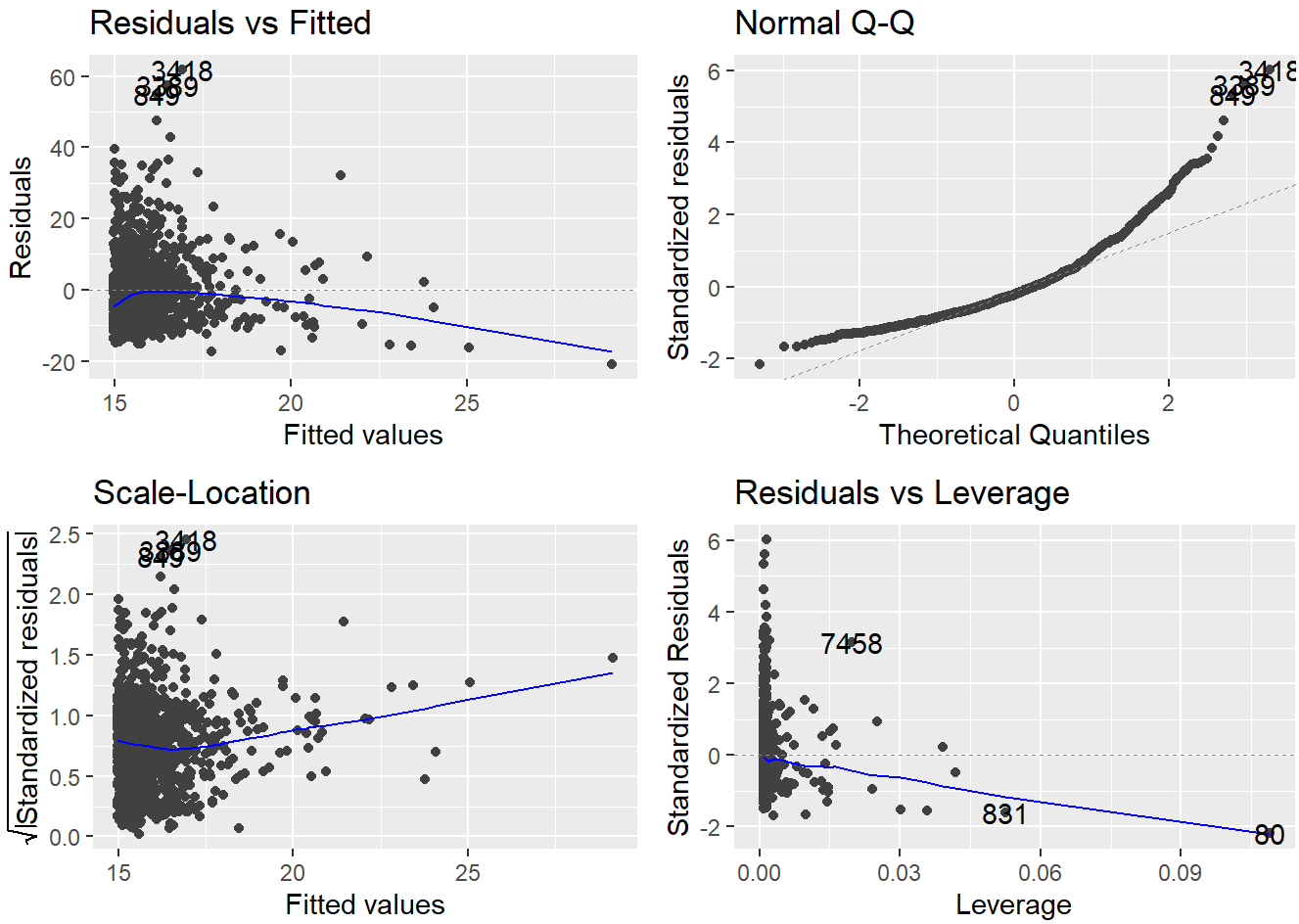



```
lin_model <- lm(DR1TFIBE ~ DR1TVD, data = modified3_full_df)

library(ggfortify)
```

```
## Warning: package 'ggfortify' was built under R version 4.5.2
```

```
autoplot(lin_model)
```




```
shapiro.test(residuals(lin_model))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(lin_model)
## W = 0.88741, p-value < 2.2e-16
```

The linearity assumptions are not met in this data set (Shapiro-Wilk normality test has p-value less than 0.05), so we will not fit a linear curve to show on the graph as well. What the graph does tell us, however, is that there is not an important relationship between vitamin D consumption and total fiber consumption, among both depressed individuals and people who do not meet criteria for clinical depression, at least according to these sample data. Fiber and vitamin D are both positive additions to a diet (meaning more, to a certain extent, is generally better), yet they are not related very strongly, as this plot suggests.

4. Build any 2 machine learning models that use your choice of covariates to predict the given outcome variable. Explain why you chose those covariates and interpret model performance for each model. Be sure to include the confusion matrix for each model, one AUROC plot with a line for each model, as well as the values of AUC for each model. If you are including categorical variables as covariates in a `glmnet` model (lasso or ridge regression), please remember that you will need to convert your covariate data frame into a *model matrix*, e.g., by calling the `model.matrix` function:

```
library(splitstackshape)
library(dplyr)
library(tree)
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
##     lift
```

```
#model_matrix = model.matrix(~.-1, my_covariate_df)

#setting up machine learning
class(modified2_full_df$DPQ020)
```

```
## [1] "numeric"
```

```
my_outcome_as_factor <- modified2_full_df %>%
  mutate(depression_dx = as.factor(DPQ020))
tiny_df <- my_outcome_as_factor %>%
  select("SEQN", "DR1TVD", "DPQ100", "DR1TFIBE", "DR1TSUGR", "DR1TSFAT", "DR1TCHOL", "DPQ020",
"depression_dx")
rownames(tiny_df) <- 1:nrow(tiny_df)
class(my_outcome_as_factor$depression_dx)
```

```
## [1] "factor"
```

```
head(tiny_df)
```

```
##    SEQN DR1TVD DPQ100 DR1TFIBE DR1TSUGR DR1TSFAT DR1TCHOL DPQ020 depression_dx
## 1 73561   23.5      1     12.3    87.78    4.479       41      1             1
## 2 73564    5.7      0     24.8   128.23   25.989      152      1             1
## 3 73567    0.3      0      7.0    98.01   28.438       81      1             1
## 4 73571    3.6      1     14.9    61.42   14.153       80      1             1
## 5 73600   16.9      3     10.9   290.30   30.553      752      2             2
## 6 73615    3.2      1     11.2    68.24   11.356      224      1             1
```

```
#copied the provided code from HW 5 here to set up ML
set.seed(1)
x <- stratified(tiny_df, "depression_dx", 0.7, keep.rownames = TRUE)
summary(x)
```

```
##       rn                 SEQN           DR1TVD           DPQ100      
##  Length:782         Min.   :73561   Min.   : 0.000   Min.   :0.0000  
##  Class :character   1st Qu.:76355   1st Qu.: 1.000   1st Qu.:0.0000  
##  Mode  :character   Median :78853   Median : 2.800   Median :0.0000  
##                     Mean   :78731   Mean   : 4.297   Mean   :0.6167  
##                     3rd Qu.:81261   3rd Qu.: 5.400   3rd Qu.:1.0000  
##                     Max.   :83724   Max.   :64.500   Max.   :9.0000  
##                                     NA's   :50       NA's   :2       
##     DR1TFIBE         DR1TSUGR         DR1TSFAT         DR1TCHOL      
##  Min.   : 0.000   Min.   :  1.36   Min.   :  0.486   Min.   :   0.0  
##  1st Qu.: 8.775   1st Qu.: 56.05   1st Qu.: 14.488   1st Qu.: 121.8  
##  Median :13.500   Median : 95.56   Median : 22.319   Median : 201.0  
##  Mean   :15.839   Mean   :114.13   Mean   : 26.227   Mean   : 273.7  
##  3rd Qu.:20.100   3rd Qu.:147.58   3rd Qu.: 33.349   3rd Qu.: 365.8  
##  Max.   :78.800   Max.   :979.39   Max.   :146.206   Max.   :1760.0  
##  NA's   :50       NA's   :50       NA's   :50        NA's   :50      
##      DPQ020      depression_dx
##  Min.   :1.000   1:632        
##  1st Qu.:1.000   2:150        
##  Median :1.000                
##  Mean   :1.192                
##  3rd Qu.:1.000                
##  Max.   :2.000                
##
```

```
train_set <- x %>% dplyr::select(-rn)
train_index <- as.numeric(x$rn)
test_set <- tiny_df[-train_index,]
test_set_y <- as.factor(test_set$depression_dx)

dim(train_set)
```

```
## [1] 782   9
```

```
dim(test_set)
```

```
## [1] 335   9
```

```
#realizing that knn won't run with any missing data, will run median imputation, as data is nonn
ormal
#copying code from slide 706 (of 806 total slides for the semester)
train_x <- train_set %>% dplyr::select(-c(depression_dx, DPQ020))
train_y <- train_set$depression_dx

test_x <- test_set %>% dplyr::select(-c("depression_dx", "DPQ020"))
test_y <- test_set$depression_dx

imputation_model <- preProcess(train_x, method = "medianImpute")

train_x_imp <- predict(imputation_model, train_x)
test_x_imp <- predict(imputation_model, test_x)

#step 5: reattach predictors and outcome
final_imputed_train <- train_x_imp %>%
  mutate(depression_dx = train_y)
final_imputed_test <- test_x_imp %>%
  mutate(depression_dx = test_y)

#machine learning model 1: k nearest neighbors
knn_fit <- knn3(depression_dx ~ ., data = final_imputed_train, k = 12)
knn_fit
```

```
## 12-nearest neighbor model
## Training set outcome distribution:
##
##   1   2
## 632 150
```

```
p_hat_knn <- predict(knn_fit, newdata = final_imputed_test, type = "prob")[,2]
summary(p_hat_knn)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00000 0.08333 0.16667 0.18423 0.25000 0.58333
```

```
y_hat_knn <- ifelse(p_hat_knn > 0.5, "Depressed", "Not Clinically Depressed")
y_hat_knn_edited <- ifelse(y_hat_knn == "Depressed", 1, 2)
cm_knn <- confusionMatrix(
  data = as.factor(y_hat_knn_edited),
                          reference = final_imputed_test$depression_dx,
  positive = "1")
cm_knn
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2
##          1   1   0
##          2 270  64
##
##                Accuracy : 0.194
##                  95% CI : (0.1531, 0.2405)
##     No Information Rate : 0.809
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0014
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.003690
##             Specificity : 1.000000
##          Pos Pred Value : 1.000000
##          Neg Pred Value : 0.191617
##              Prevalence : 0.808955
##          Detection Rate : 0.002985
##    Detection Prevalence : 0.002985
##       Balanced Accuracy : 0.501845
##
##        'Positive' Class : 1
##
```

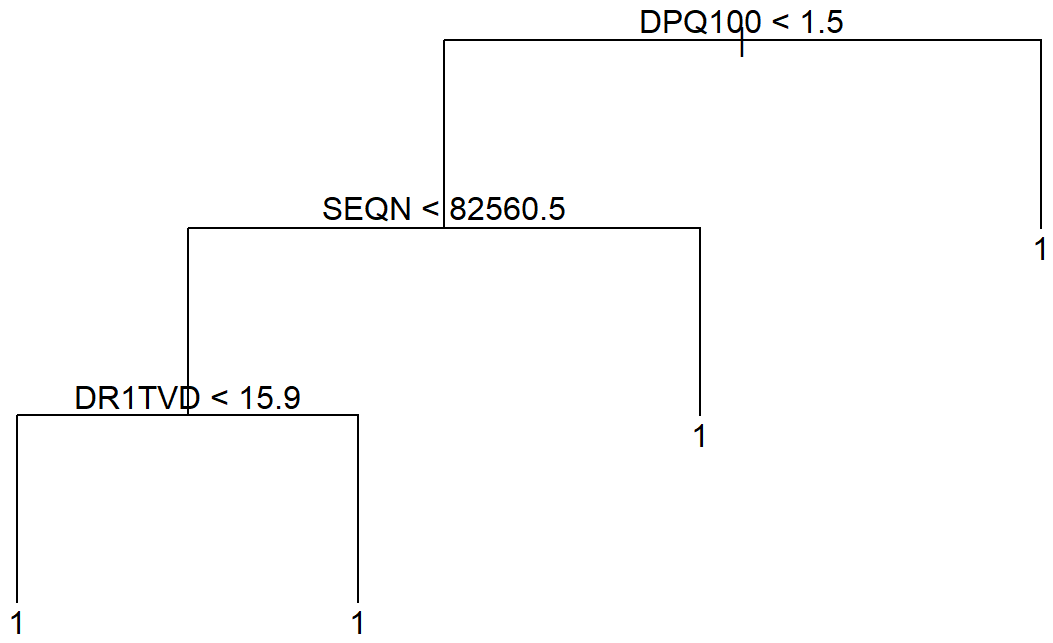
```
#see next chunk for machine learning model #2
```

Accuracy of the k-nearest neighbors model is 0.194 (95% CI: 0.1531, 0.2405), which is worse than guessing. This indicates that the model classifying almost everyone as not depressed when some individuals in fact have depression. An enormous number of people (off the main diagonal in the confusion matrix) fall into the category with 270 individuals, who do experience depression despite the model misclassifying these individuals as unaffected. The perfect specificity (100%) and the fact that in the testing data, 271 people have depression versus 64 unaffected individuals, means that probably there are additionally errors in the splitting of training and testing data, as the training and testing sets should each have approximately equal numbers of depressed and not depressed survey participants, since that is the function of the “splitstackshape” package. In the actual survey data, only a minority of people experienced clinical depression, in contrast to the majority in this testing data set, indicating a problem.

```
#machine learning model 2: classification tree
tree_fit = tree(depression_dx ~ .,
                data = final_imputed_train)
summary(tree_fit)
```

```
##
## Classification tree:
## tree(formula = depression_dx ~ ., data = final_imputed_train)
## Variables actually used in tree construction:
## [1] "DPQ100" "SEQN"   "DR1TVD"
## Number of terminal nodes:  4
## Residual mean deviance:  0.924 = 718.9 / 778
## Misclassification error rate: 0.1918 = 150 / 782
```

```
plot.new()
plot(tree_fit, type = "uniform")
text(tree_fit, pretty = 0)
```



```
p_hat_tree <- predict(tree_fit,
                      newdata = final_imputed_test)[,2]
#p_hat_tree
y_hat_tree <- ifelse(p_hat_tree > 0.5, "Depressed", "Not Clinically Depressed")
#y_hat_tree
y_hat_tree_edited <- ifelse(y_hat_tree ==
                              "Depressed", 1, 2)
cm_tree <- confusionMatrix(
  data = as.factor(y_hat_tree_edited),
  reference = final_imputed_test$depression_dx,
  positive = "1")
```

```
## Warning in confusionMatrix.default(data = as.factor(y_hat_tree_edited), :
## Levels are not in the same order for reference and data. Refactoring data to
## match.
```

```
cm_tree
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2
##          1   0   0
##          2 271  64
##
##                Accuracy : 0.191
##                  95% CI : (0.1503, 0.2373)
##     No Information Rate : 0.809
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.000
##             Specificity : 1.000
##          Pos Pred Value :   NaN
##          Neg Pred Value : 0.191
##              Prevalence : 0.809
##          Detection Rate : 0.000
##    Detection Prevalence : 0.000
##       Balanced Accuracy : 0.500
##
##        'Positive' Class : 1
##
```

The classification tree model, according to the confusion matrix, failed to classify anyone as affected by depression using all the variables available to it (although only DPQ100, extent to which depression is impacting daily life and making it difficult; SEQN, the participant's identification number; and DR1TVD, the total daily intake of vitamin D on

the first day of the interview series were actually used by the model). In other words, the inflated specificity value of 100% (perfect classification) and the extremely low value for sensitivity (0%) means that model is essentially "dumping" all people into the "not depressed" category without regard for their individual covariate values and reaping the "benefit" of getting approximately 50% (balanced accuracy = 50%) of people correctly classified as a side effect of doing this "dumping" procedure. The overall accuracy is 0.191 (95% CI: 0.1503, 0.2373). This overall, when combined with the conclusions drawn after running the k-nearest neighbors model, means that probably there is an overarching issue with the data set or the training/testing split, not with the code used to run the models themselves.

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
#model 1: k nearest neighbors
roc_knn <- roc(final_imputed_test$depression_dx,
               p_hat_knn)
```

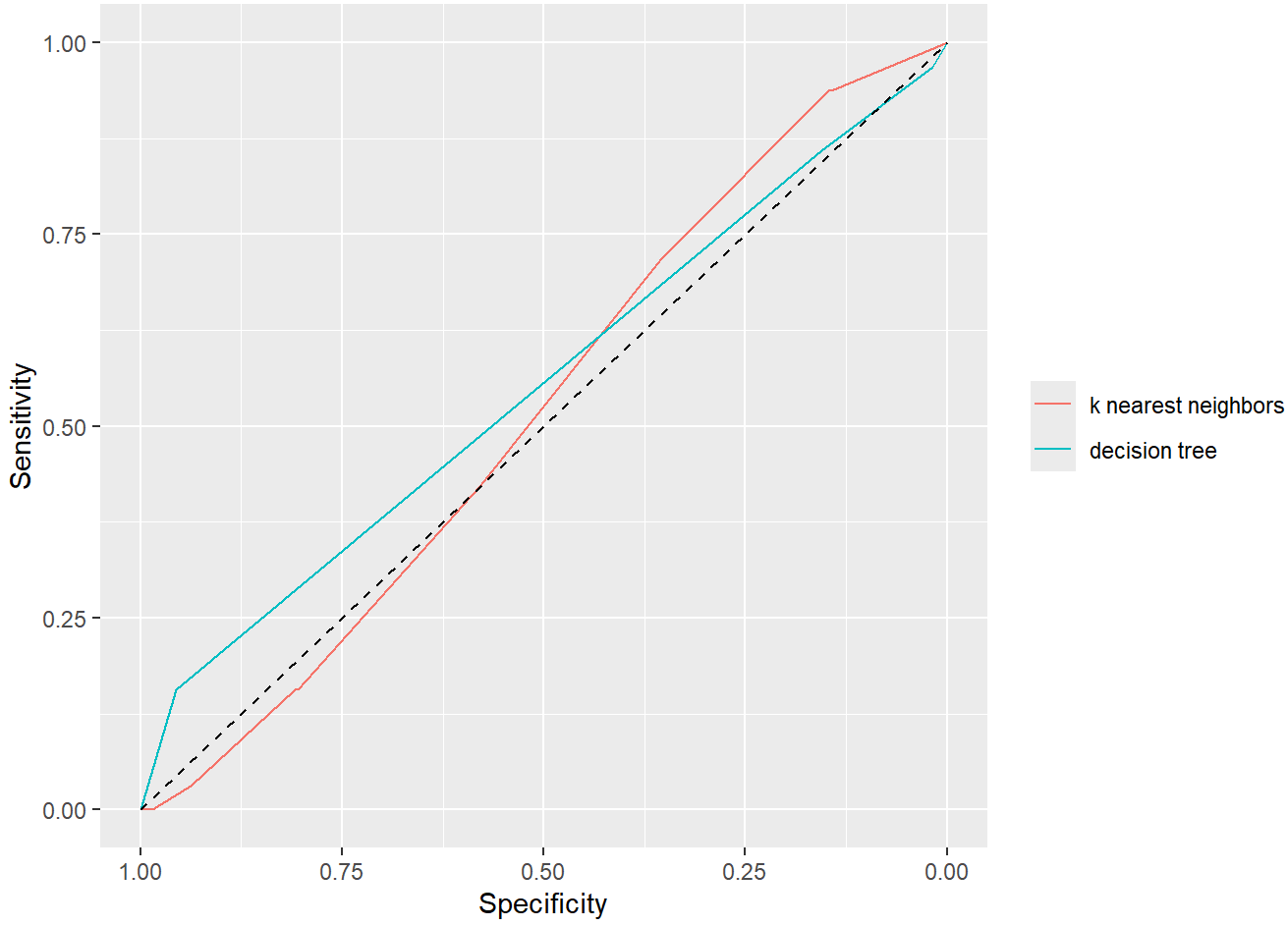
```
## Setting levels: control = 1, case = 2
```

```
## Setting direction: controls < cases
```

```
#model 2: classification tree
roc_tree <- roc(final_imputed_test$depression_dx,
                p_hat_tree)
```

```
## Setting levels: control = 1, case = 2
## Setting direction: controls < cases
```

```
#plotting both ROC on same graph
ggroc(list("k nearest neighbors" = roc_knn, "decision tree" = roc_tree)) +
  theme(legend.title = element_blank()) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1),
               color = "black",
               linetype = "dashed") +
  ylab("Sensitivity") +
  xlab("Specificity")
```




```
#model 1: knn model
auc(roc_knn)
```

```
## Area under the curve: 0.5209
```

```
#model 2: decision tree
auc(roc_tree)
```

```
## Area under the curve: 0.553
```

5. Provide a summary of your findings/conclusions/recommendations in a way that people with a non-technical background (like the CEO and CMO) can understand.

Based on our findings, there is insufficient evidence to conclude that various aspects of diet (total sugar, total saturated fat, total cholesterol, daily intake of vitamin D, and total fiber content) can predict a depression diagnosis. A major limitation of the models run here is that they vastly over predict people as not having depression when in fact they do. If medical professionals wanted to draw conclusions between dietary status and depression diagnosis status, they would need to perform further research, perhaps incorporating more variables that are more closely linked to depression (such as anxiety or major life events, which are not data included in NHANES).

6. Is there any additional information or data that might be useful to collect/have for answering your prompt?

After accounting for the limitations of the models run in this file, other data that would be helpful for predicting depression might be other mental health factors such as an anxiety level, an average stress level, and knowledge about major life events (such as birth, divorce, or death in the family). It appears, based on this preliminary exploration of the data, that dietary factors do not influence or predict a diagnosis of clinical depression well at all.

NHANES (2013-2014)

The National Health and Nutrition Examination Survey (NHANES) assesses the health and nutritional status of adults and children in the United States. The `covariate_df` data frame in the `covariate_df.rData` file contains the demographic, examination, diet, and lab data from NHANES 2013-2014 (https://www.kaggle.com/datasets/cdc/national-health-and-nutrition-examination-survey). You can peruse the dictionaries for the variables on the CDC website: demographics (https://wwwn.cdc.gov/Nchs/Nhanes/Search/variablelist.aspx?Component=Demographics&CycleBeginYear=2013), examinations (https://wwwn.cdc.gov/Nchs/Nhanes/Search/variablelist.aspx?Component=Examination&CycleBeginYear=2013), diet (https://wwwn.cdc.gov/Nchs/Nhanes/Search/variablelist.aspx?Component=Dietary&CycleBeginYear=2013), and lab (https://wwwn.cdc.gov/Nchs/Nhanes/Search/variablelist.aspx?Component=Laboratory&CycleBeginYear=2013). **Your goal is to use these covariates to predict the occurrence of diabetes, cancer, heart attack, stroke, OR depression. The company wants you to recommend a predictive model and identify key variables that appear to associated with the outcome disease.**

As part of data processing, choose ONE of the five diseases (diabetes, cancer, heart attack, stroke, depression) and merge the corresponding variable from the `questionnaire.csv` dataset with `covariate_df`. A binary variable indicating if a person has this disease is the outcome that your model should predict. The relevant variable names in `questionnaire.csv` are: DIQ010 (diabetes), MCQ220 (cancer), MCQ160E (heart attack), MCQ160F (stroke), and DPQ020 (depression). (You can view the entire questionnaire dictionary here (https://wwwn.cdc.gov/Nchs/Nhanes/Search/variablelist.aspx?Component=Questionnaire&CycleBeginYear=2013).) For each disease variable, you can take `1` to be affected, `2` to be unaffected, and all other numeric values to be unknown/ignored (i.e., remove all rows with any values not equal to 1 or 2 for the disease you choose).